

From Raw Data to Real Predictions

An end-to-end data pipeline, from a messy dataset to a model you would deploy.

FORMAT: Notebook only

GROUP SIZE: 20 students

SUBMISSION: Google Form

MAX MARK: 110

1. The Task

You have joined the data team of a company that has a dataset and a hunch. They believe patterns are hiding in it that could change how they treat customers, price products, or allocate resources, but nobody is sure what those patterns are. They want two things: an honest map of the natural groups in the data, and a model that predicts the outcome they care about.

Work as a team, run your assigned dataset through the full pipeline you learned this term, and submit one notebook that tells the whole story. Every group must complete every stage. You cannot skip clustering, classification, or imbalance handling; each is graded on its own.

2. Your Dataset

Each group is pre-assigned one Kaggle dataset (four groups share each one, but no two arrive at the same answer). Find your group number below. Download the data inside the notebook with `kagglehub` so it reproduces on a fresh Colab session; a direct Kaggle URL will not work without an API token. If your dataset has more than 100,000 rows, you may use a stratified sample: state the size and `random_state` and justify it.

Group 01	Credit Risk / Loan Default	Group 14	Telco Customer Churn
Group 02	IBM HR Attrition	Group 15	Credit Card Churn
Group 03	Adult Census Income	Group 16	Telco Customer Churn
Group 04	IBM HR Attrition	Group 17	Insurance Cross-Sell
Group 05	Credit Card Churn	Group 18	Bank Marketing
Group 06	Hotel Booking Cancellations	Group 19	Credit Risk / Loan Default
Group 07	Heart Failure Prediction	Group 20	Heart Failure Prediction
Group 08	Adult Census Income	Group 21	IBM HR Attrition
Group 09	Insurance Cross-Sell	Group 22	Stroke Prediction
Group 10	Bank Marketing	Group 23	Adult Census Income
Group 11	Adult Census Income	Group 24	Bank Marketing
Group 12	IBM HR Attrition	Group 25	Bank Marketing
Group 13	Hotel Booking Cancellations	Group 26	Credit Risk / Loan Default

Group 27	Insurance Cross-Sell	Group 34	Stroke Prediction
Group 28	Stroke Prediction	Group 35	Credit Risk / Loan Default
Group 29	Credit Card Churn	Group 36	Telco Customer Churn
Group 30	Stroke Prediction	Group 37	Telco Customer Churn
Group 31	Credit Card Churn	Group 38	Heart Failure Prediction
Group 32	Heart Failure Prediction	Group 39	Insurance Cross-Sell
Group 33	Hotel Booking Cancellations	Group 40	Hotel Booking Cancellations

WATCH FOR LEAKAGE. Two datasets contain columns that give the answer away and must be dropped before modeling. **HOTEL BOOKING:** drop `reservation_status` and `reservation_status_date`. **CREDIT CARD CHURN:** drop the two `Naive_Bayes_Classifier` columns at the end of the file (they are another model's output).

Dataset links

Dataset	Kaggle page
Bank Marketing (Term Deposit)	kaggle.com/datasets/janiobachmann/bank-marketing-dataset
IBM HR Analytics Attrition	kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset
Adult Census Income	kaggle.com/datasets/uciml/adult-census-income
Stroke Prediction	kaggle.com/datasets/fedesoriano/stroke-prediction-dataset
Hotel Booking Cancellations	kaggle.com/datasets/jessemotipak/hotel-booking-demand
Credit Risk / Loan Default	kaggle.com/datasets/laotse/credit-risk-dataset
Heart Failure Prediction	kaggle.com/datasets/fedesoriano/heart-failure-prediction
Insurance Cross-Sell	kaggle.com/datasets/anmolkumar/health-insurance-cross-sell-prediction
Credit Card Customer Churn	kaggle.com/datasets/sakshigoyal7/credit-card-customers
Telco Customer Churn	kaggle.com/datasets/blastchar/telco-customer-churn

3. Deliverables

Structure the notebook in these seven parts, using the exact headings shown. The weight of each part is noted beside it.

Part 1. Data Foundation (15%)

1. Load the dataset from Kaggle inside the notebook and push it into a SQLite database.

2. Run at least three SQL queries that reveal something useful, each followed by a markdown cell explaining what you learned.
3. Perform EDA: shape, types, missing values, target distribution, and three to five feature distributions.
4. Clean the data, justifying every step in markdown. No blanket `fillna`. If you drop rows, explain why.

Part 2. Feature Engineering and Encoding (20%)

1. Create at least three new features (for example: bin a continuous column, count occurrences per row, take a ratio, or build an interaction term). Each needs a markdown cell explaining the business reasoning.
2. Encode every categorical column, choosing label or one-hot for each and justifying it. Use `drop_first` where appropriate and say why.
3. Scale the features, choosing between `StandardScaler` and `MinMaxScaler`, and justify the choice.

Part 3. Clustering Analysis (20%)

1. Run K-Means for K from 2 to 10. Plot the elbow curve and the silhouette scores.
2. Pick a final K and justify it using both signals.
3. Profile each cluster with group means and give it a plain-English name (e.g. "long-tenure budget customers").
4. Visualize the clusters in two dimensions using PCA.

Part 4. Classification (20%)

1. Set up the task with `train_test_split` and `stratify=y`.
2. Train at least three classifiers from those covered in class, justifying each choice.
3. Report accuracy, precision, recall, F1, and ROC AUC for each, with a confusion matrix, and collect them in one comparison table (models in rows, metrics in columns).
4. If your dataset has fewer than 2,000 rows, use stratified k-fold cross-validation instead of a single split and report the mean and standard deviation of each metric.

Part 5. Imbalance Handling (10%)

1. Apply at least two of: class weights, threshold tuning, SMOTE.
2. For threshold tuning, plot the precision-recall curve and pick a threshold tied to a stated business cost.
3. Show a table of baseline versus each technique. If your dataset is roughly balanced (40 to 60 percent positive), still apply the techniques, but show with numbers that they change little and explain why. You are graded on the reasoning.

Part 6. The Tie-In (5%)

1. Compute the positive-class rate within each K-Means cluster from Part 3.
2. Were any clusters strong predictors of the label? Explain what that tells you about the data.

Part 7. Conclusion and Recommendations (10%)

One markdown section answering, specific to your data and results: which model you would deploy and why; what you would do with another month; what the clusters revealed that the classifier missed; and the hardest decision your team made.

Depth Track (10 bonus marks)

On top of the full pipeline, pick **one** track and go deeper. Draw conclusions from it, not just output.

- **Clustering:** compare K-Means against hierarchical or DBSCAN (report agreement via crosstab or adjusted Rand index), test stability on three 80% subsamples, and build three named personas with representative rows.
- **Classification:** train four or more classifiers with cross-validation, plot all ROC curves on one chart, and compare feature importances across models.
- **Imbalance:** apply all three techniques, plot precision-recall curves, and recommend a final operating point with cost reasoning.

4. Grading Rules

Marks follow the weights above. The following are hard requirements and deductions applied before grading.

Requirements

- The notebook runs end to end on a fresh Colab session (Restart and run all, no errors). We will run it.
- Every cleaning, engineering, encoding, scaling, and modeling decision has a markdown sentence above the code explaining why.
- Your final model beats a stratified dummy (`DummyClassifier(strategy='stratified')`) on both positive-class F1 and ROC AUC, and that row appears in your comparison table. On an imbalanced dataset you will likely need threshold tuning to clear this.
- Your final ROC AUC is at least 0.75. Every dataset in the pool can reach this.
- If any model scores near-perfect (close to 1.0), stop and find the leaking column, say what it was, and drop it. A perfect score is a bug.

Deductions

Marks off	Reason
up to -10	Notebook does not run end to end on a fresh Colab session
-5	A leaking column left in the model (its score is meaningless)
-3	Missing Submission Summary table, or renamed part headings
-2	Full run takes longer than 20 minutes on a free Colab CPU runtime

5. Submission

The **first markdown cell** of your notebook must be this Submission Summary table, filled in. Everyone in a group submits the same notebook under their own name and receives the same score.

Group number	07
Your full name	Ada Okoro
Dataset	Heart Failure Prediction
Depth track	Classification Depth
Team members	Full names of all members
Final model	Random Forest, class_weight balanced, threshold 0.42
Stratified dummy F1 / ROC AUC	0.44 / 0.50
Final model F1 / ROC AUC	0.88 / 0.93
Leakage columns dropped	None found, or list them
Full run time on Colab	6 minutes

Steps:

1. Name the file `group_NN_yourname.ipynb` (zero-padded group number), e.g. `group_07_ada_okoro.ipynb`.
2. Save it with all outputs visible.
3. Submit through the capstone form: [the capstone submission form](#).
4. Keep the confirmation the form shows you. That is your proof of submission.

BEFORE YOU SUBMIT, CONFIRM: the notebook restarts and runs clean; the Submission Summary is the first cell; the seven part headings plus the Depth Track heading are present and correctly named; the stratified dummy is in your table and beaten on both metrics; your ROC AUC is at least 0.75; and no model scores near 1.0.

A strong story beats a fancy model. Think carefully, justify your choices, and good luck.